

Design of a REST API

Prof. Dr. Peter Braun

6. Januar 2025

1 Titlepage

Design of a REST API

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- REST constraints by Roy Fielding
- REST and HTTP: technical aspects
- REST development methodology

Welcome to this unit, where we will explore the REST paradigm and a methodology for developing interfaces based on this paradigm. We will examine the REST architectural style from two perspectives. We will begin with the REST architectural constraints outlined in the PhD thesis of Roy Fielding. Then we will proceed to some technical aspects of the HTTP protocol as one example of REST. Each of these technical elements is addressed in a separate unit within this lecture series, so think of this technical overview as either an introduction or a summary of the other units. Finally, we will discuss a suggested methodology for integrating REST into your work as a team of software developers.

3 REST Constraints (Fielding, 2000)

REST Constraints (Fielding, 2000)

- ❶ Client-server
- ❷ Statelessness
- ❸ Cacheability
- ❹ Uniform interface
- ❺ Layered system
- ❻ Code on Demand (optional)

In his doctoral thesis, Roy Fielding describes two approaches to designing a software system. The first approach begins with existing components. Similar to a modular system, these components are combined to create a complex software system from the bottom up. The system is adjusted until all requirements are satisfied. The second approach starts with the system requirements in their entirety, initially without any constraints. Constraints are then identified step by step and applied to the system's components. Roy Fielding used this second approach in his doctoral dissertation. The six constraints beside me ultimately led to the architectural style known as REST.

4 Client-Server

Client-Server

- Client and Server exchange messages
- It's always the client that starts the communication
- Advantage: Separation of Concerns

The first limitation arises from a client-server system. The server offers a specific service that clients can access. For the client to use the service, it must send a message to the server, which processes this message and responds to the client. Communication is always initiated by the client. A significant advantage of client-server systems is the principle of Separation of Concerns. For example, the client manages the user interface presentation, while the server takes care of business logic and database operations. This clear division of responsibilities enables the interchangeability of components. A peer-to-peer system differs from a client-server system and is therefore not REST-compliant.

5 Statelessness

Statelessness

- Server does not know about clients' state
- Advantage: Scalability
- Disadvantage: Network load

The second constraint relates to statelessness. Specifically, this means the server does not store state information about clients locally through sessions. Clients must always send all necessary information with each request so the server can process it. This approach offers advantages in terms of scalability, as the server does not need to retain any state information about clients between requests. However, the downside is that more data needs to be transmitted.

6 Cacheability

Cacheability

- Messages can be **cacheable** or **non-cacheable**.
- Caches are independent components

The third limitation relates to caches. Each response message from the server must be marked as cacheable or non-cacheable. If a response message is allowed to be cached, the server grants the cache the authority to release this message during subsequent requests, following established rules, without the need to consult the server again. This approach can lead to the server receiving significantly fewer requests in an optimal scenario, which benefits scalability and performance. However, a downside of caching is that clients may receive outdated data, which is typically a design issue with the caching rules. It is also REST-compliant to disable caching from the server; however, a system should be prepared for a cache to be positioned between a client and a server, even without the knowledge of either party.

7 Uniform Interface

Uniform Interface

- Small set of operations
- Unique identifiers
- Representations
- Self-descriptive messages
- Hypermedia as the engine of application state

The fourth constraint is the Uniform Interface, which sets REST apart from other distributed system styles. It indicates a limited set of operations with clearly defined semantics. This limited set of operations is sufficient to meet most if not all, programming needs of a distributed system. The same applies to response messages regarding success or failure in processing requests. With this limitation, we assume that resources or documents are exchanged between the client and server. These resources have addresses and can have various representations. Self-describing messages are exchanged between the client and server, meaning neither party needs additional data beyond what is included in the messages to interpret them. In addition to data, these messages may include meta information, such as the date of the last update. The concept of hyperlinks, used by the server to inform the client of allowable next steps, also falls under this constraint. An example counter to self-describing messages is a protocol that constructs complex data structures at the byte level. In such cases, the client and server must know precisely where the attribute "firstName" begins within the byte stream. This additional specialized knowledge is something to be avoided in REST.

8 Layered System

Layered System

- Layers only communicate with neighbors
- Clients do not know how many layers exist

The fifth constraint is referred to as the Layered System. In this system, individual layers are organized, and communication is limited to adjacent layers. A message from the client, on its way to the server, may pass through several systems, with each layer capable of modifying the message, which is inherently descriptive. The client cannot determine how many layers a message traverses before processing. For instance, one might picture these layers in such a way that one layer handles authentication, the next oversees business logic, and the third is responsible for data storage, with even a cache coming before that. In contrast, a non-layered system would feature an architecture where, for example, the client must decide which components to send a request to or which specific layers need to be bypassed during communication.

9 Code on Demand (optional)

Code on Demand (optional)

- Clients load code from the server
- Example: JavaScript

The sixth and final constraint is called Code On Demand, which is optional. Not every system adhering to the REST architectural style is required to implement this constraint. Code On Demand allows the client to reload program code from the server as needed. This approach reduces client complexity and enables features that are rarely needed to load only when required. Consequently, the range of functions available to a client can also expand at runtime. Roy Field references Java applets as an example in his doctoral thesis. However, this technology is now considered outdated. In contrast, JavaScript serves as a modern example.

10 REST

REST

■ Representational State Transfer

Let's reevaluate REST, or Representational State Transfer, and its meaning. Clients manage their state, providing information for servers to process requests without needing to know clients' recent resource access. The client sends this state to the server, which modifies it and returns relevant data. For instance, to find all Persons named James, the client asks the server to search and receives the first ten results in the response. This cycle continues as the client updates data and communicates with the server, illustrating state transfer. Representational means data is sent with a specific representation to meet self-describing message requirements. Systems adhering to the 5 + 1 constraints may be REST compliant, but not all use HTTP entirely. Some comply with REST without HTTP, and not all HTTP systems are REST compliant. For example, a remote function call using HTTP may violate the unified interface constraint and thus be non-REST compliant, like peer-to-peer systems or server-managed client states through sessions.

11 Resources, Identifiers, and Representations

Resources, Identifiers, and Representations

- Resources model the real world
- URIs as unique identifiers
- Data is sent as representation: media types

Next, we examine the practical implementation of a distributed application using REST architecture with HTTP. This overview addresses resource representation, which is essential in REST systems. Resources, which represent data such as information about a person or a book, are accessed through unique URIs or URLs. Data is expressed not as raw binary but in detailed formats that ensure clients and servers can correctly interpret the content. REST-compliant representations always include attribute names and relevant metadata, such as the last update date, with this information divided between the header and content areas of the message. The format of the content area is known as the media type. Resources can have multiple representations, including JSON or XML, and may also be provided as images or graphics, depending on the use case. The server may offer different attributes based on the requester's role; for instance, administrators may access more data than regular users.

12 Associations

Associations

- Resources are related to each other
- One person has many tasks
- One building has many rooms

Resources are interconnected. Much like a database in which relations exist between entities, resources are also logically linked. For instance, tasks can be assigned to individuals, or a building can consist of multiple rooms. Depending on the nature of these connections, various techniques are employed in a REST-compliant system. For example, resources can be nested within one another or linked through hyperlinks.

13 CRUD

CRUD

- Create
- Read
- Update
- Delete

A fundamental requirement for a REST-compliant system is the proper application of what are known as CRUD operations. This directly aligns with the constraints of the unified interface. There is a minimal set of operations: reading, writing, updating, and deleting resources. These four operations are adequate to manage all interactions with resources in a distributed system. The CRUD operations correspond to the four HTTP verbs: GET, POST, PUT, and DELETE. The semantics of these HTTP verbs are clearly defined in the specifications, and both the server and client must adhere to them. For example, the GET operation must not alter any data on the server. All operations, except for POST, must be idempotent, meaning they can be executed multiple times in succession without altering the result each time. Many distributed systems built on the HTTP protocol effectively implement these principles to the fullest extent. They model their data as resources and appropriately utilize CRUD operations. However, compliance with the principles of security and idempotency can sometimes be less precise. Nevertheless, even if one adheres to those principles, a significant issue remains: without observing the following principle, a distributed system should not assert that it is REST compliant.

14 Hypermedia

Hypermedia

- Server provides hyperlinks to the next states
- Clients (or end users) pick one of them
- Only one single URI as the entry point

The hypermedia property is significant. This means that the server notifies the client in its response message about the permitted operations. The client or end-user chooses a subsequent action from this offer, thus advancing the application's state. This principle can be somewhat difficult to grasp and even more challenging to implement in practice. It requires effort and, most importantly, a certain degree of courage on the part of the server developer to anticipate the allowed states within the application and to avoid permitting anything beyond that. Let's consider a few examples. In a REST-compliant system, the client needs only a single entry address provided by the server. All other request messages are directed only to those addresses that the server has previously communicated. The client never constructs an address independently. The process of constructing addresses is entirely unimportant and should remain fully transparent to the client's developer. On the other hand, the server developer retains full freedom in deciding how to construct the addresses and does not need to document this process in advance for the client's developer. If you are still struggling with the idea that the server must always suggest all possible follow-up actions to the client, consider the following example. Suppose the client requests a list of people sorted by last name. If the server allows the sort order to be reversed, it must provide the client with a corresponding address in the form of a hyperlink in its response message, which the client must call in the next step to achieve this result. How the server encodes the sort criterion into this address is secondary at first. What is essential is that the client does not need to understand this encoding. Or consider the following example as well. Suppose the server wants to allow the currently active end-user to modify the requested resource. In that case, the server includes a hyperlink in its response message to the GET request, indicating the operation to the client. Only if this hyperlink is included in the response message can the client offer the

end-user, for example, a button to modify the resource. If this hyperlink is not included, the client must not provide such an action. At the same time, the server must also ensure that only authorized users can initiate the PUT operation.

15 Pagination

Pagination

- Page Number
- Offset-Size
- Cursor-based

The number of resources provided by the server in a GET request can become quite extensive. The client can use two techniques to limit the number of results or view only a subset of them. The first technique, known as pagination, employs three different methods that allow the client to load specific pages or sections of a large set of resources. The choice of which method to use is determined solely by the server, leaving the client without the option to select any method independently.

16 Filtering and Sorting

Filtering and Sorting

- Server provides simple query parameters
- Don't implement a complex query language
- URI-Templates
- Default order – hyperlinks to change it

A related issue is the search and sorting of the result set. By using query parameters, the client can search for resources based on specific attribute values, thereby narrowing the result set. Such searches should be straightforward, and the server should be tolerant of potentially incorrect values, meaning it should still return results. A frequently asked question is how to perform complex searches, including those involving logical or comparison operators. Unfortunately, it is common practice for an API to provide an SQL-like language for formulating complex search queries. However, this practice violates the principle of maintaining a uniform interface, as a separate complex language disrupts uniformity. In a REST-compliant system, the server must clearly define what the search query should entail and communicate this to clients through URI templates embedded in hyperlinks. The same principle applies to sorting order. In a REST-compliant system, the server specifies which attributes can be used for sorting and whether switching between ascending and descending order is allowed. All of this is presented to the client as hyperlinks.

17 Caching

Caching

- Cache Topology
- Expiration model using validity period
- Validation model using timestamps or Etags

A key limitation of REST-compliant systems is caching. Due to REST's statelessness, substantial data must be exchanged between the client and server with each request, as the server cannot retain any client information. This increases network load, which caching can help alleviate. Requests may pass through various caches—one directly at the client, another shared by the internet service provider, or one within the service provider's network. Each cache can temporarily store responses and return them automatically for new requests, reducing the need to contact the server. To prevent data inconsistency, caches depend on the server for information, using HTTP protocol metadata to manage cache behavior. The server establishes validity periods, permitting caches to release records immediately during that time. Once the period lapses, caches may need to confirm changes with the server. This verification can employ timestamps, where the client notifies the server of the last retrieval date for comparison. If the record is updated, the client receives the new version; otherwise, a brief message confirms possession of the latest version. An alternative method utilizes hash values instead. Caching helps reduce network load, particularly for frequent read requests, where infrequently changing data is cached locally more efficiently than data that changes often.

18 HTTP Security

HTTP Security

- Transport Layer Security
- HTTP Basic Authentication using ID and password
- HTTP Bearer Authentication using tokens

Finally, we discuss the topic of HTTP safety. On the one hand, we must consider transport security. Messages between the client and server should always be encrypted, if possible, to protect them from eavesdropping and manipulation. Additionally, we need to address how to restrict resource access for known and registered end-users, a process referred to as authentication. We can also associate access with user groups or roles, known as authorization. We will explore two techniques for authentication and authorization. In the first method, the client transmits the end-user's ID and password to the server with each request. In the second method, the ID and password are converted into a token during the initial request, and all subsequent accesses utilize only the token. The advantage of this second approach is that a token can include a validity period. If, contrary to expectations, a token is compromised or overheard, the attacker can only misuse it for a limited time.

19 Development Methodology

Development Methodology

- ❶ Decide on the architectural style
- ❷ Discuss your understanding of REST
- ❸ Restrict the degree of freedom of REST
- ❹ Model the resources
- ❺ Model the end-users and user roles
- ❻ Define your media types
- ❼ Decide on authentication and authorization
- ❽ Decide on the caching strategies
- ❾ Decide on paging, filtering, and sorting
- ❿ Implement Hypermedia thoroughly!

To develop a distributed system in a project, first consider whether you want to follow REST principles. There are alternatives, such as remote function calling, which lacks a single interface since it offers numerous functions. Developers must understand these functions through documentation. Another difference is that, in function-oriented systems, servers can cache client states, unlike REST's stateless nature. When choosing between alternatives, consider both technical and human factors; if the team is experienced with certain methods, it is reasonable to use them. However, growth in technical skills might be beneficial depending on the budget. If the team opts for REST, assess their understanding because REST principles are complex and not easily checked in code. Although many frameworks support REST development, developers can still overlook REST specifications. Achieving a shared understanding of REST principles is crucial, and decisions on authentication, pagination, and caching should be unified before commencing the project. Once clarified, gather requirements and start by modeling data, not functions, as REST limits operations to reading, creating, updating, and deleting data. After modeling the data, consider user roles regarding data access and tailor views for specific user groups. Adjust media types if necessary. Plan caching strategies carefully, taking data validity and source into account, and only later decide on filtering and sorting options. Ensure that the hypermedia principle of REST is upheld during implementation, meaning servers must provide hyperlinks to subsequent states.

20 What's still open?

What's still open?

- Documentation using OpenAPI or Swagger
- Automated testing
- Development of REST-ful clients

As I mentioned, this collection includes separate units that cover all these topics. However, there are some topics that, unfortunately, we cannot address here due to time constraints. I want to focus on three topics. While we have often emphasized that documenting a REST interface is unnecessary due to the unified interface and self-describing messages, the client or client developer can find all the essential data either by using the interface or relying on behavior according to the HTTP specification. Nonetheless, it is common for a REST interface to be documented in a basic manner. Tools like OpenAPI or Swagger serve this purpose. A notable feature of these tools is that they can automatically generate client source code. The second topic we cannot address here is automated testing. One consequence of the hypermedia property is that the client only needs to know a single entry point on the server. This complicates testing because, in each test case, if implemented correctly, numerous requests must always be executed, starting from the initial state to the state currently being tested. Additionally, automated testing of caches, for example, can be quite costly due to the potentially long validity period of resources. Finally, we have not addressed the issue of programming a web-based user interface in any unit. Today's web frameworks, such as Vue.js and Angular, do not inherently support the hypermedia property. Much of this functionality must be implemented manually, and the effort required is considerable.

21 Summary

Summary

- The REST constraints of Roy Fielding
- HTTP as one example of REST
- REST Development Methodology
- Have fun with REST!

So, that's it for today. We approached the topic of REST from two perspectives. First, we followed the path laid out by Roy Fielding, the creator of the term REST, in his PhD thesis. Starting with the overall system requirements, we established constraints and presented the 5 + 1 constraints that Fielding outlined in his thesis. This should help you gain a better understanding of the REST acronym. On the other hand, we explored the technology behind the HTTP protocol, discussing its features and implications. We covered resources, addresses, the four CRUD operations, the fundamental hypermedia principle, techniques for reducing result sets, strategies for minimizing network load, and methods for user authentication and authorization. As I mentioned, there are separate units in this collection that examine each of these technical topics in greater depth, along with numerous programming examples to facilitate understanding. Finally, on a more abstract level, we discussed a possible methodology for using REST as an architectural style to program distributed applications. Enjoy studying and applying REST principles to distributed systems.

22 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0 – Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.
- Jim Webber, Savas Parastatidis, Ian Robinson: REST in Practice – Hypermedia and Systems Architecture. O'Reilly, 2010.
- Roy Thomas Fielding: Architectural Styles and the Design of Network-based Software Architectures. PhD Thesis, University of California, Irvine, 2000.